SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

ASSIGNMENT-1

QUESTIONS

CO2: Analyze public-key cryptosystem strategies using RSAandkeyexchange for encryption algorithms to strengthen information security.

Assignment Title: Evaluate, Design and Defend a Secure Public-Keyand Hybrid Cryptographic Strategy for a Multi-Site Healthcare Network(Total Marks: 50) Scenario — Secure Healthcare Data Exchange

Code of Conduct:

I **P.Sunayana(192424303)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

1.Introduction, Scope and Method

a. The scenario in one paragraph

The healthcare network contains one central hospital and multiple remote clinics. They exchange patient records, prescriptions, diagnostic reports and billing information over public networks. Some clinics use low-resource devices, while the network must also support large file transfers and increasing numbers of sites. The design must protect confidentiality, establish session keys securely, authenticate senders, detect modification and remain suitable for future cryptographic changes.

b. What the assignment asks

This report evaluates symmetric algorithms and block-cipher modes, builds the number-theory foundations required for RSA, examines RSA attacks and padding, and compares Diffie–Hellman with elliptic-curve cryptography. It then combines signatures and hybrid encryption into a deployable design and revisits that design under the Day-4 challenge, where traffic volume, weak key generation, man-in-the-middle attacks and future quantum computing are considered.

c. Method and evidence boundary

The worked arithmetic examples in this report use small, student-created toy values so that every result can be checked by hand. Pseudocode and Python-style core routines are supplied to support implementation. Toy examples demonstrate mathematical behaviour only; they are not secure parameters and must never be used to protect real patient data. No live timing measurements or external execution logs were supplied with the question paper. Therefore, operation counts below are analytical counts or predicted trends, not claimed benchmark measurements. Actual execution time should be recorded on the student's computer.

d. Notation

Symbol	Meaning
P, C, K	Plaintext, ciphertext and cryptographic key.
n, e, d	RSA modulus, public exponent and private exponent.
□(n)	Euler totient of n.
p, g, a, b	Diffie–Hellman prime modulus, generator and private exponents.
G, Q	ECC base point and public point.
IV / nonce	Initialization vector / number used once.
H(M)	Cryptographic hash of message M.

e. Report roadmap

Q1 compares symmetric algorithms and modes. Q2 develops modular arithmetic and prime

selection. Q3 demonstrates RSA and its weaknesses. Q4 covers Diffie–Hellman, authentication and key management. Q5 compares ECC with RSA and finite-field DH. Q6 designs signatures and hybrid encryption and explains quantum readiness. Q7 integrates the findings into a revised architecture.

1. Q1 – Symmetric Cryptosystem Selection and Modes of Operation (7 Marks)

1.1 Algorithm comparison

A symmetric cipher uses the same secret key, or a directly related key, for encryption and decryption. It is suitable for bulk clinical records because symmetric operations are much faster than public-key operations.

Algorithm	Structure and block size	Key characteristics	Round operation / assessment
DES	16-round Feistel network; 64-bit block.	56 effective key bits (64 supplied bits including parity).	Expansion, XOR with round key, 8 S-boxes, permutation. Exhaustive key search is practical today; obsolete.
3DES	DES applied three times; 64-bit block.	Two-key or three-key variant; effective security is limited by meet-in-the-middle attacks; 64-bit block is small.	Common EDE form: encrypt-decrypt-encrypt. Much slower than one DES; legacy only.
Blowfish	16-round Feistel network; 64-bit block.	Variable key length 32–448 bits; key setup is relatively expensive.	Key-dependent P-array and four S-boxes; fast after setup, but 64-bit block limits modern high-volume use.
RC5	Parameterized block cipher; block size is 2w bits (commonly 64 or 128).	Variable word size, key length and number of rounds.	Data-dependent rotations, modular addition and XOR. Flexible, but security depends on parameters and implementation.

1.2 Comparison and selection

For this legacy-algorithm comparison, the practical ranking is: (1) Blowfish or a suitably configured RC5 for legacy study use, (2) 3DES only where compatibility forces it, and (3) DES last. This ranking does not mean Blowfish or RC5 should be selected for a new healthcare deployment. Both Blowfish and 3DES use 64-bit blocks, creating birthday-bound collision concerns at high data volumes. RC5 is parameter-dependent and less commonly deployed. DES has an inadequate key size. A modern authenticated cipher such as AES-GCM or ChaCha20-Poly1305 is preferable for production.

Criterion	Block cipher	Stream cipher
Processing	Transforms fixed-size blocks; padding or a mode may be needed.	Generates a keystream combined with plaintext, usually by XOR.
Typical use	Files, database records and storage when used with a secure mode.	Continuous or low-latency data streams and constrained communication.
Main caution	Mode and IV/nonce selection are essential.	Never reuse a keystream/nonce with the same key.

1.3 Block cipher modes

Mode	Operation and IV/nonce rule	Pattern leakage / errors / parallelism
ECB	Encrypts each block independently; no IV.	Equal plaintext blocks produce equal ciphertext blocks and reveal patterns. Blocks encrypt/decrypt in parallel. Avoid.
CBC	XOR each plaintext block with previous ciphertext; random unpredictable IV required.	Hides repeated blocks when IV is fresh. One corrupted ciphertext block damages its plaintext block and flips corresponding bits in the next block. Encryption sequential; decryption parallel.
CFB	Cipher output is fed back; IV required and unique/unpredictable as specified.	Turns block cipher into a self-synchronizing stream-like mode. Errors affect current and following segment(s). Encryption sequential.

OFB	Repeatedly encrypts feedback state to generate keystream; unique IV required.	Bit error affects only corresponding plaintext bit. Keystream can be generated in advance; encryption/decryption parallel after generation.
CTR	Encrypts nonce plus counter to form keystream; nonce-counter pairs must never repeat under a key.	No block pattern leakage with unique nonce; bit errors affect corresponding bits only. Fully parallel and suitable for high throughput.

1.4 ECB pattern and nonce-reuse demonstration

Suppose a test file contains plaintext blocks A, B, A, C. ECB produces ciphertext blocks E(A), E(B), E(A), E(C). The first and third ciphertext blocks are identical, revealing that the plaintext blocks repeat even though the attacker cannot read them. CBC with a fresh random IV produces different chaining inputs for repeated A blocks, so the resulting ciphertext blocks are normally different.

For CTR, ciphertext is P XOR S, where S is the keystream generated from the key and nonce/counter. If a nonce is reused, the same S is generated. Then C1 XOR C2 = P1 XOR P2. This relation can reveal information about both plaintexts and may allow recovery when one plaintext is known. Therefore, a nonce must never repeat with the same key.

1.5 Core simulation and expected tests

```
# Demonstrate ECB pattern leakage using a deterministic toy
block mapping. # This is a teaching simulation, not a real cipher.
toy_encrypt = {"A": "X7", "B": "Q2", "C": "M9"}
plain_blocks = ["A", "B", "A", "C"]
cipher_blocks = [toy_encrypt[x] for x in
plain_blocks] print(cipher_blocks)  # ['X7', 'Q2',
'X7', 'M9'] print(cipher_blocks[0] ==
cipher_blocks[2])  # True


# CTR nonce-reuse relation (toy
bytes) p1, p2, stream = 0x41,
0x52, 0xA7
c1, c2 = p1 ^ stream, p2 ^ stream
print(hex(c1 ^ c2), hex(p1 ^ p2))  # equal values
```

For the required implementation evidence, run at least three available cipher implementations on

synthetic strings or files, decrypt each result and assert that the recovered plaintext equals the input. Record the algorithm, key length, message size, encryption time, decryption time and operation count. Do not label toy substitution mappings as DES, 3DES or Blowfish implementations.

Message size	Expected operation trend	What to record
1 KB	Small fixed setup cost may dominate.	Encryption/decryption time and number of blocks.
100 KB	Work increases approximately with block count.	Blocks processed, total time and throughput.
1 MB	Bulk symmetric processing remains practical.	Average of repeated runs; exclude first-run setup if reported separately.
10 MB	Time should scale roughly with bytes for a fixed algorithm/mode.	Throughput in MB/s and verification result.

1.6 Evaluation

For the healthcare network, legacy DES and 3DES should not be selected for new protection. Blowfish and RC5 are useful for comparative study but their deployment suitability is limited by legacy status, parameter concerns and (for common configurations) 64-bit blocks. Use a modern authenticated block mode such as AES-GCM for files, or a modern stream-based AEAD such as ChaCha20-Poly1305 for devices where it is efficient. Authentication tags are essential because encryption alone does not detect malicious modification.

2. Q2 – Number-Theory Engine and Prime Selection for RSA (6 Marks)

2.1 GCD and Extended Euclidean Algorithm

The greatest common divisor gcd(a,b) is the largest positive integer dividing both a and b. Euclid's algorithm repeatedly replaces (a,b) with (b, a mod b) until the remainder becomes zero. The Extended Euclidean Algorithm also finds integers x and y such that ax + by = gcd(a,b).

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def extended_gcd(a,
    b): if b == 0:
        return a, 1, 0
    g, x1, y1 = extended_gcd(b, a
    % b) return g, y1, x1 - (a // b)
    * y1

def mod_inverse(a, m):
    g, x, y = extended_gcd(a,
    m) if g != 1:
        raise ValueError("Inverse does not
    exist") return x % m
```

2.2 Modular inverse and Euler's totient

The modular inverse of a modulo m is a value x satisfying ax □ 1 (mod m). It exists exactly when gcd(a,m)=1. For RSA, choose distinct primes p and q, set n=pq and □(n)=(p□1)(q□1). Choose public exponent e with 1<e<□(n) and gcd(e,□(n))=1. The private exponent d is the modular inverse of e modulo □(n), so ed □ 1 (mod □(n)).

Euler's totient □(n) counts the positive integers up to n that are relatively prime to n. For n=pq with distinct primes,

□(n)=(p□1)(q□1).

2.3 Fast modular exponentiation

RSA requires powers modulo n. Repeated multiplication takes a number of multiplications proportional to the exponent. Square-and-multiply uses the binary representation of the exponent and requires O(log e) modular multiplications.

```
  def mod_pow(base, exp,
      mod): result = 1
      base %=
      mod while
      exp > 0:
          if exp % 2 == 1:
              result = (result * base) %
          mod base = (base * base) % mod
          exp //= 2
      return result
```

2.4 Hand-checkable RSA key example

Choose p=61 and q=53. Then n=61×53=3233 and □(n)=60×52=3120. Choose e=17 because gcd(17,3120)=1. The inverse is d=2753 because 17×2753=46801=1+15×3120. Thus the public key is (3233,17) and the private key is (3233,2753).

Value	Calculation	Result
p, q	Selected distinct primes	61, 53
n	61 × 53	3233
□(n)	60 × 52	3120
e	gcd(17,3120)=1	17
d	17 × 2753 □ 1 mod 3120	2753

2.5 Prime testing methods

Method	Basic idea	Strength and limitation
Trial division	Test divisibility by primes up to □n.	Simple and deterministic for small values; impractical for large candidates.
Fermat test	For a base a, test a^(n□1) □ 1 (mod n).	Fast but some composites pass; Carmichael numbers fool every coprime

		base.
Miller–Rabin	Write n□1=2^s·d with d odd; test one or more bases.	Efficient probabilistic test. Repeated independent bases make false acceptance probability very small for suitable random bases.

Carmichael numbers such as 561, 1105 and 1729 are composite numbers that pass Fermat's congruence for every base relatively prime to the number. A Fermat-only prime generator can therefore accept composites. Miller–Rabin is preferred for practical RSA prime generation because it is efficient and has a well-understood error bound when bases are selected correctly.

2.6 Prime density and scaling

The prime number theorem estimates the probability that a large random integer near N is prime as approximately 1/ln(N). Therefore, the expected number of random odd candidates tested before finding a prime grows roughly with ln(N), not exponentially in the bit length. Candidates are first filtered by small-prime trial division and then tested with Miller–Rabin.

RSA modulus target	Engineering interpretation	Decision
Classroom toy	Tiny values permit manual factoring and hand calculations.	Demonstrates mathematics only; insecure.
1024-bit	No longer recommended for new long-term protection.	Do not deploy for healthcare records.
2048-bit	Common minimum baseline for conventional RSA deployments.	Use only with current approved padding and vetted libraries.
3072-bit	Provides a larger classical security margin than 2048-bit RSA.	Consider where longer-term classical security is required; still not post-quantum.

The operation-count comparison should record: Euclidean division iterations, modular multiplications in square-and-multiply, number of trial divisors, Miller–Rabin rounds and candidate rejections. A timing comparison must use the same machine, software and repeated runs. The expected result is that Extended Euclid and square-and-multiply scale much better

than naive search and repeated multiplication.

2.7 Engineering decision

Use a cryptographically secure random-number generator, reject repeated or too-small primes, ensure p and q are distinct and sufficiently separated, use approved primality tests, and protect private-key material. The mathematical core can be implemented for learning, but production key generation should use a vetted cryptographic library and current standards.RSA operation
RSA encryption maps an integer message M with 0□M<n to C=M^e mod n. Decryption computes M=C^d mod n. The correctness follows from ed□1 mod □(n), together with the RSA number-theory properties for properly generated keys.

2.8 Hand-worked encryption and decryption

Using the key pair from Q2, take M=65. Encryption gives C=65^17 mod 3233=2790. Decryption gives 2790^2753 mod 3233=65. The original message is recovered. This confirms correctness for the selected toy input, but does not prove that textbook RSA is secure.

Item	Value
Plaintext integer M	65
Public key (n,e)	(3233,17)
Ciphertext C	2790
Private exponent d	2753
Decrypted integer	65

2.9 Required tests and predictions

Test	Prediction	Expected observation / conclusion
Encrypt same M twice with textbook RSA	Same ciphertext each time.	RSA is deterministic without randomized padding; repeated messages are linkable.
Small message and small exponent	Some small messages may be recovered if M^e<n.	Textbook RSA has algebraic structure; low-exponent and no-padding use is unsafe.

Test	Prediction	Expected observation / conclusion
Boundary M=0	Ciphertext is 0 and decryption returns 0.	Mathematically valid but not a meaningful secure encoded message.
Boundary M=n	Invalid because RSA message representative must satisfy 0□M<n.	Reject out-of-range input.
e not coprime to □(n)	No modular inverse d exists.	Key generation must reject this parameter.
p=q or non-prime factors	RSA assumptions fail.	Reject weak or invalid key material.

2.10 Factoring attack on a weak toy modulus

If an attacker factors n into p and q, the attacker calculates □(n)=(p□1)(q□1) and then computes d=e■¹ mod □(n). The private key is therefore recovered. For the toy modulus n=3233, trial division finds 53×61. Once the factors are known, the private exponent 2753 can be derived. This is why RSA security depends on the difficulty of factoring a properly generated large modulus.

Fermat factorisation writes n=a²□b²=(a□b)(a+b) and searches for a such that a²□n is a square. It is effective when p and q are close. For example, n=101×103=10403 has a=102 and b=1, giving factors 101 and 103 immediately. If one factor is very small, trial division finds it quickly. These classroom demonstrations do not imply that modern 2048-bit RSA can be factored by the same small search.

2.11 Why key length alone is not enough

- Poor random-number generation can produce predictable or repeated primes.
- Reusing a prime across keys or selecting primes with weak structure can expose private keys.
- Incorrect padding, side-channel leakage, timing differences and implementation bugs can defeat an otherwise large modulus.
- Textbook RSA is deterministic and malleable; it must not be used directly for application messages.
- Private keys must be protected, access-controlled, backed up securely and revoked after compromise.

2.12 OAEP padding

RSA-OAEP (Optimal Asymmetric Encryption Padding) encodes a message using a random seed and mask-generation function before RSA exponentiation. The randomization makes encryptions of the same message produce different encoded representatives and ciphertexts. OAEP also provides a structured encoding check during decryption. A modern implementation

must use a suitable hash and mask-generation function and must follow the approved RSA-OAEP parameter profile. OAEP does not make RSA suitable for encrypting arbitrary large files; it is used to protect short keys or small messages.

2.13 Attack trace and conclusion

```
# Toy factor
recovery n = 3233
for p in range(2, int(n**0.5) +
    1): if n % p == 0:
        q = n // p
        print("factors:", p, q)  # 53,
        61 phi = (p - 1) * (q - 1)
        print("phi:", phi)         # 3120
        break
```

Prediction: the toy modulus will be factored almost immediately and the private exponent will be recoverable. Evidence: the factors 53 and 61 give □(n)=3120, and the inverse of 17 modulo 3120 is 2753. The experiment is only a demonstration of the mathematical attack; classroom-sized factors are vastly smaller than engineering RSA keys.

3. Q3 – Diffie–Hellman Key Exchange, Authentication and Key Management (7 Marks)

3.1 derivation

The two clinics agree publicly on a large prime p and a generator g. Clinic A chooses private value a and sends A=g^a mod p. Clinic B chooses private value b and sends B=g^b mod p. A computes K=B^a mod p=(g^b)^a mod p=g^(ab) mod p. B computes K=A^b mod p=(g^a)^b mod p=g^(ab) mod p. Both obtain the same shared secret without sending a or b.

3.2 Pseudocode

Public parameters: prime p,
generator g Clinic A:
choose secret
a A = pow(g,
a, p)
send A to Clinic
B Clinic B:
choose secret
b B = pow(g,
b, p)
send B to Clinic A
Clinic A computes K = pow(B, a,
p) Clinic B computes K = pow(A,
b, p)
Both derive session_key = KDF(K, transcript, context)

3.3 Small hand-verifiable example

Let p=23 and g=5. Clinic A chooses a=6 and sends A=5^6 mod 23=8. Clinic B chooses b=15 and sends B=5^15 mod 23=19. A computes 19^6 mod 23=2. B computes 8^15 mod 23=2. The shared value is 2. This small group is intentionally insecure and is used only to check the arithmetic.

Parameter	Clinic A	Clinic B
Private exponent	a=6	b=15
Public value	A=8	B=19
Shared computation	19^6 mod 23	8^15 mod 23
Shared result	2	2

3.4 Larger toy configuration and complexity

For a larger toy prime, the same procedure is used with a larger p and longer exponents. Modular exponentiation by square-and-multiply requires O(log p) modular multiplications when exponents are of the same bit length as p. A brute-force discrete-log attacker may test candidate exponents until g^x matches the public value; the search grows with the group size. Real deployments use standardized groups and vetted implementations rather than classroom-sized primes.

3.5 Man-in-the-middle attack

Unauthenticated Diffie–Hellman does not prove who sent a public value. An attacker Mallory intercepts A and sends her own public value to B; she also intercepts B and sends a different public value to A. A establishes a key with Mallory, and B establishes another key with Mallory. Mallory can decrypt, read, modify and re-encrypt traffic between them while each clinic believes it shares a key with the other.

A -> Mallory:
A_public Mallory -
> B: M1_public B -
> Mallory: B_public
Mallory -> A:
M2_public
A and Mallory derive K_AM
B and Mallory derive K_MB
Mallory relays messages after decrypting and re-encrypting.

The defence is authentication. Each clinic signs the ephemeral DH public value and the handshake transcript with its certified signing key. The peer validates the certificate chain, identity, validity period, revocation status and signature. Binding both public values and the transcript prevents substitution. A mutually authenticated TLS configuration is a practical example.

3.6 Key distribution and management policy

Lifecycle stage	Policy
Generation	Generate private exponents using a cryptographically secure random generator; use approved groups and reject invalid public values.
Storage	Keep long-term private keys in a hardware security module or protected keystore; restrict access by role.
Distribution	Use a certificate authority and authenticated, encrypted channels. Never email or hard-code private keys.
Rotation	Rotate certificates and long-term keys according to policy; create fresh ephemeral session keys for each connection.
Revocation	Publish and check revocation status; immediately disable compromised credentials.
Compromise recovery	Revoke key/certificate, issue new credentials, investigate logs and re-establish sessions.
Session lifetime	Use short-lived session keys; rekey on time, data-volume threshold or security event.

3.7 Evaluation

Diffie–Hellman solves the problem of establishing a shared secret over an observable network, but unauthenticated DH is vulnerable to MITM. Authentication through certificates and digital signatures is mandatory. The session secret should be passed through a key-derivation function rather than used directly as an encryption key.

4. Q4 – ECC and Public-Key Strategy Comparison (5 Marks)

4.1 Mathematical basis of ECC

Elliptic-curve cryptography uses points on a curve over a finite field. A common curve form is $y^2 \equiv x^3+ax+b \pmod p$, where p is prime and the curve is non-singular. The public key is Q=dG, where d is a private integer and G is a publicly specified base point. The security problem is the elliptic-curve discrete logarithm problem: given G and Q.

4.2 Toy ECC key agreement

For a classroom illustration, use the curve $y^2=x^3+2x+2 \pmod{17}$ and base point G=(5,1), whose order is 19. Let Alice choose private dA=5 and Bob choose dB=7. Their public keys are QA=5G=(9,16) and QB=7G=(0,6). Alice computes 5QB=35G and Bob computes 7QA=35G. Since $35G \equiv 16G \pmod{19}$, both obtain the same point. This toy curve is far too small for security.

Item	Alice	Bob
Private scalar	5	7
Public point	5G = (9,16)	7G = (0,6)
Shared point	5 × QB = 35G	7 × QA = 35G
Result	Same shared point	Same shared point

4.3 RSA, finite-field DH and ECC comparison

Feature	RSA	Finite-field DH	ECC / ECDH
Security basis	Integer factorisation.	Discrete logarithm in a finite field.	Elliptic-curve discrete logarithm.
Public material	Large modulus and exponent.	Group parameters and public group element.	Curve parameters and point.
Typical size comparison	Larger keys for comparable classical security.	Usually larger public values than ECC.	Smaller keys and lower bandwidth for comparable classical security.
Main use	Encryption with OAEP; signatures.	Key agreement; not direct bulk encryption.	ECDH key agreement; ECDSA signatures.

Feature	RSA	Finite-field DH	ECC / ECDH
Deployment	Broad compatibility; RSA operations can be costly on constrained devices.	Needs validated group and authenticated exchange.	Efficient for constrained devices, but requires correct curve and implementation.

4.4 Implementation requirements and recommendation

• Use secure random generation for private scalars and ephemeral values.

• Validate received points and parameters; reject invalid, off-curve or small-subgroup points as required by the chosen protocol.

• Protect private keys in hardware-backed or operating-system-protected storage.

• Use standard named curves and vetted libraries; do not design a new curve for this assignment.

• Authenticate ECDH values with certificates or signatures to prevent MITM.

For low-resource clinic devices, ECDH with a modern standardized curve is recommended because public-key material and bandwidth are smaller than traditional RSA-style parameters. Use it inside an authenticated protocol such as TLS 1.3, with a signature-based certificate identity. RSA can remain for compatibility where needed, but it should not be used for bulk-file encryption. The recommendation depends on secure implementation and key management, not merely on shorter key size.

5. Q5– Digital Signatures, Hybrid Encryption and Quantum-Cryptography Readiness (7 Marks)

5.1 Hash-then-sign digital signature

A digital signature provides sender authentication, integrity and evidence that the holder of a private signing key approved a message. The sender calculates a cryptographic hash H(M) and signs the digest using the private signing key. The receiver uses the sender's public key to verify the signature against the received message digest. The receiver must also validate the certificate that binds the public key to the sender's identity.

Sender:

```
digest = Hash(message)
signature = Sign(sender_private_key, digest)
send(message, signature, certificate)
```

Receiver:

```
verify certificate and sender identity
digest2 = Hash(received_message)
accept only if Verify(sender_public_key, digest2, signature)
```

5.2 Signature algorithm comparison

Algorithm	Keys and operation	Randomness / considerations
RSA signatures	RSA public/private key pair; modern signature padding such as RSA-PSS is used.	PSS uses a salt and secure hash; verification is generally efficient. Do not confuse RSA-PSS signatures with RSA-OAEP encryption.
DSA / DSS	Finite-field discrete-log signature scheme.	Requires a unique, secret per-message nonce. Reuse or leakage of the nonce can reveal the private key. Legacy choice for new systems.
ECDSA	Elliptic-curve public/private key pair.	Also critically depends on a unique secret nonce per signature. Deterministic nonce generation can reduce randomness failure risk when correctly implemented.

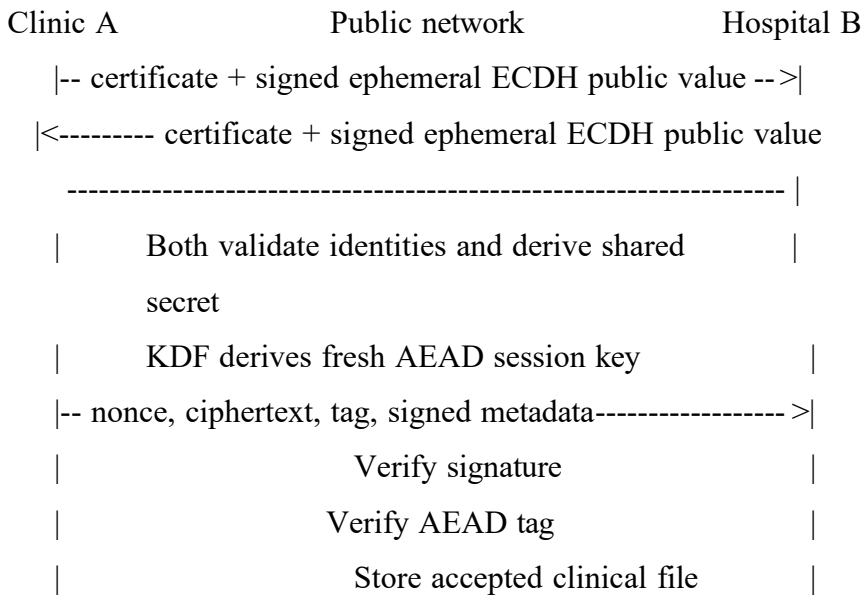
5.3 Hybrid encryption design

Public-key encryption is inefficient for large files and has strict message-size limits. Hybrid encryption uses a random symmetric session key for the file and public-key cryptography only to

establish or protect that short key. A recommended authenticated design is:

- a fresh random session key for each file or transfer session.
- Encrypt the large file using an authenticated symmetric cipher such as AES-256-GCM or ChaCha20-Poly1305, with a unique nonce.
- Establish the session key using authenticated ephemeral ECDH, or protect a random session key with RSA-OAEP where compatibility requires it.
- Sign the file metadata, sender identity, recipient identity, algorithm identifiers and transcript using a digital signature.
- Send the ciphertext, nonce, authentication tag, signature and certificate chain.
- The receiver validates the certificate and signature, derives or unwraps the session key, verifies the authentication tag and only then releases the plaintext.

5.4 Hybrid message flow



5.5 Tamper tests

Test	Expected result	Security property
Change one ciphertext bit	AEAD tag verification fails; plaintext is rejected.	Integrity and confidentiality protection.
Change one signature bit	Signature verification fails.	Sender authenticity and integrity.
Replace protected/derived session key	Key confirmation or AEAD tag check fails.	Key agreement and message authentication.

Replay an old prescription	Reject using a sequence number, timestamp or unique transaction ID tracked by receiver.	Freshness and replay protection.

5.6 Direct RSA versus hybrid encryption

Direct RSA encryption of a large payload is not practical: RSA can encrypt only a short encoded message whose size is limited by the modulus and padding. A large file would need to be split into many small blocks, causing many expensive public-key operations and complicating integrity protection. Hybrid encryption performs one key agreement or one key-wrapping operation per session and uses fast symmetric processing for all file blocks.

Measure	Direct RSA approach	Hybrid approach
Public-key operations	Potentially many operations and usually not suitable for large files.	One authenticated key agreement or one key-wrap per session.
Bulk data	RSA cannot directly process arbitrary large files efficiently.	Symmetric AEAD processes the complete file efficiently.
Integrity	Requires a separate signature/MAC construction.	AEAD tag plus optional sender signature.
Size overhead	Padding and block framing repeated; strict size limits.	Small key/nonce/tag/signature metadata plus ciphertext approximately file-sized.

5.7 Quantum key distribution and quantum threats

Quantum key distribution (QKD) uses quantum states to help two parties establish shared key material. In protocols such as BB84, measuring unknown quantum states can disturb them; after comparing a sample of measurement results over an authenticated classical channel, the parties can estimate whether eavesdropping has occurred. If the estimated error rate is too high, they abort. QKD requires specialised hardware and a trusted/authenticated classical channel. It does not authenticate users by itself, encrypt application data, manage certificates, prevent endpoint

compromise or replace ordinary key management.

A sufficiently capable fault-tolerant quantum computer running Shor's algorithm would threaten RSA factoring and the discrete-log problems used by conventional DH and ECC. A readiness plan should inventory cryptographic dependencies, use crypto-agile interfaces, follow standardised post-quantum algorithms as they become approved, test hybrid classical-plus-post-quantum key establishment, update certificates and protocols, and prioritise data that must remain confidential for many years. Grover's algorithm also affects symmetric brute-force security more modestly; using appropriately sized symmetric keys provides a margin.

5.8 Deployment decision

The recommended design is authenticated ephemeral ECDH for session establishment, AEAD symmetric encryption for bulk files, and digital signatures for sender authentication and auditability. Use RSA-OAEP only where compatibility requires key transport, and RSA-PSS or ECDSA for signatures according to the selected certificate ecosystem. For long-term readiness, design protocol agility and pilot hybrid post-quantum key establishment. QKD may be considered only for specialised high-value links where its infrastructure is justified.

6. Q6 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

6.1 A. Diagnose the bottleneck or weakness (2 marks)

The main bottleneck in the original approach is using public-key operations directly on large files, together with the security weakness of unauthenticated key exchange. Public-key exponentiation is substantially more expensive than symmetric encryption and is constrained by padding and message size. As file size and clinic count increase, the number of public-key operations and certificate/key-management events would increase. Separately, an unauthenticated DH exchange allows a man-in-the-middle to establish two independent secrets and read or modify traffic.

6.2 B. Define the design objective (1 mark)

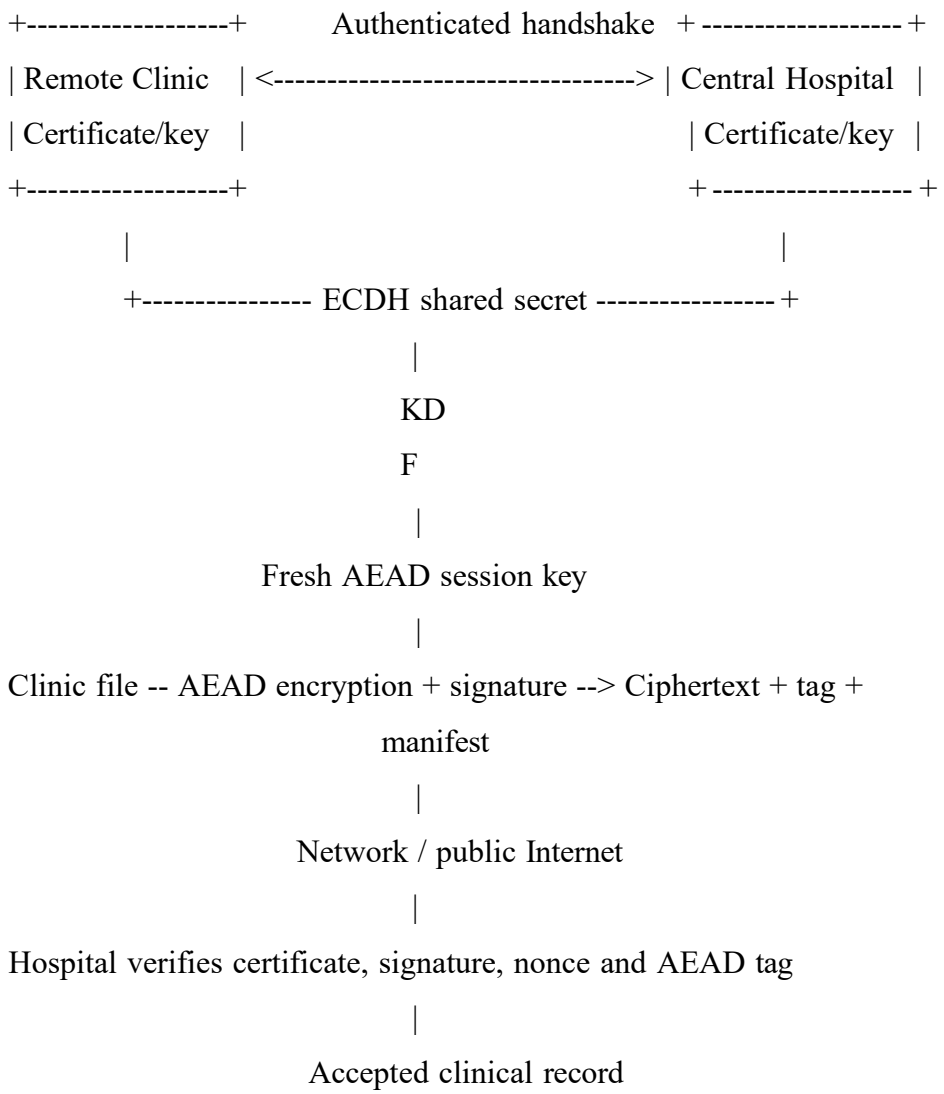
Objective: for each large clinical file, limit public-key work to one authenticated session-key establishment per session, encrypt the complete payload using an AEAD cipher, reject any tampered file or signature, and prevent unauthenticated public-key substitution through certificate/signature validation.

6.3 C. Design the revised strategy (3 marks)

The revised architecture uses authenticated ephemeral ECDH, a key-derivation function, AES-GCM or ChaCha20-Poly1305, digital signatures, certificate validation and centralised key management. Each clinic has a unique identity and certificate; there is no network-wide shared private key.

1. Clinic and hospital exchange certificates and ephemeral ECDH public keys.
2. Each side validates the certificate and verifies a signature over the handshake.
3. Both derive the same ECDH shared secret.
4. A KDF derives separate encryption and authentication/session keys.
5. The sender encrypts the file with AEAD and a unique nonce.
6. The sender signs the metadata / transfer manifest.
7. The receiver validates signature, freshness and AEAD tag.
8. The receiver stores the plaintext only after all checks succeed.
9. Keys are rotated, revoked and audited under the lifecycle policy.

6.4 Revised architecture diagram (text form)



The mathematical property used is the ECDH shared-secret equality, authenticated by signatures over the handshake transcript. AEAD combines encryption and integrity verification. A fresh session key and unique nonce prevent repeated-keystream exposure. Additional state includes certificates, revocation information, sequence numbers, nonce management and audit logs. The trade-offs are certificate infrastructure, signature verification and the need for reliable key storage and rotation.

6.5 D. Integrate quantum readiness (1 mark)

RSA, finite-field DH and ECC are vulnerable to a sufficiently capable quantum computer running Shor's algorithm. The revised design must isolate key establishment behind a crypto-agile interface and plan a transition to standardised post-quantum key encapsulation and signatures. During migration, a hybrid key-establishment exchange can combine classical ECDH with a post-quantum mechanism, deriving the final session key from both secrets. QKD can help distribute key material and detect interception on specialised links, but it does not replace endpoint authentication,

signatures, AEAD encryption or key lifecycle management.

6.6 E. Predict, implement and validate (2 marks)

Prediction: hybrid encryption will reduce public-key operations from a number that grows with file blocks to a small fixed number per session. AEAD verification will reject ciphertext modification, and signature verification will reject sender/manifest tampering. Certificate-authenticated ECDH will prevent the simple MITM substitution demonstrated in Q4.

Validation item	Original direct/unauthenticated approach	Revised approach
Public-key work for a large file	Repeated or block-wise public-key operations; inefficient and constrained.	One authenticated key agreement per session (plus signature operations).
Ciphertext tampering	May be undetected if encryption is used alone.	AEAD tag fails; reject file.
Sender identity	Unauthenticated DH does not prove peer identity.	Certificate and transcript signature bind peer identity to DH values.
Replay	Can be accepted if no freshness state exists.	Sequence number, nonce and transaction ID checked.
Large-file handling	Poor scalability for public-key encryption.	Symmetric AEAD scales with payload size.

A reproducible student test should use a synthetic file at several sizes (for example 1 KB, 100 KB, 1 MB and 10 MB). Record the number of public-key operations, symmetric bytes processed, total time, throughput, ciphertext size and verification outcome.

6.7 F. Defend the final design (2 marks)

The final design is preferred because it separates responsibilities: ECDH establishes a session secret, a KDF derives keys, AEAD protects bulk data, signatures authenticate senders and certificate management establishes trust. It is more efficient than direct RSA encryption and more secure than unauthenticated DH. It supports constrained devices through compact ECC public material and symmetric bulk processing. Residual risks include compromised endpoints, stolen private keys, certificate-authority failure, incorrect nonce handling, denial of service, implementation side channels and future cryptanalytic advances. These risks are reduced through hardware-backed keys, monitoring, revocation, secure updates, tested libraries, short session lifetimes and a post-quantum migration plan.

Q7 – Diagnose, Redesign and Validate the Complete Cryptographic Strategy (11 Marks)

Introduction

The multi-site healthcare network connects a central hospital with several remote clinics to exchange patient records, medical reports, prescriptions and other confidential information. The existing cryptographic system uses symmetric and public-key algorithms to provide security during communication.

After analysing Q1–Q6, an important limitation is identified: the high computational cost of public-key encryption for large files and the security weakness of unauthenticated Diffie–Hellman key exchange. To overcome these limitations, a revised hybrid cryptographic strategy is proposed using AES-GCM, RSA/ECC, authenticated key exchange, digital signatures and secure key management.

Q7.A – Diagnose the Bottleneck or Weakness (2 Marks)

The major computational limitation in the original system is the use of public-key encryption for large amounts of data. RSA requires more computational resources than symmetric encryption. When large medical files are encrypted, repeated public-key operations increase processing time and reduce system performance.

Another important weakness is unauthenticated Diffie–Hellman key exchange. Although Diffie–Hellman allows two parties to generate a common shared secret, it does not independently verify their identities. Therefore, an attacker can perform a Man-in-the-Middle (MITM) attack by intercepting and replacing the exchanged public values.

The original system also has the following limitations:

1. High computational cost: Repeated RSA operations increase processing time when the message size increases.
2. Authentication weakness: Basic Diffie–Hellman does not verify the communicating parties.
3. MITM vulnerability: An attacker can establish separate shared keys with the hospital and clinic.
4. Weak encryption modes: ECB mode reveals repeated plaintext patterns and does not provide data integrity.
5. Key-management issues: Reusing keys or generating weak keys can expose confidential medical information.

From Q3, RSA was demonstrated using small values $p=61$, $q=53$, $n=3233$, $e=17$ and $d=2753$. From Q4, Diffie–Hellman successfully generated a shared secret, but the absence of authentication creates a security risk. Q6 demonstrated that hybrid encryption can overcome the performance limitation by using symmetric encryption for large files.

Therefore, the original system requires improved performance, authentication and key management.

Q7.B – Define the Design Objective (1 Mark)

The main objective is to redesign the cryptographic system to provide secure, efficient and reliable communication between the central hospital and remote clinics.

The proposed design aims to:

1. Reduce the number of public-key operations during large-file encryption.
2. Prevent Man-in-the-Middle attacks through authenticated key exchange.
3. Provide confidentiality and integrity for medical records.
4. Improve secure session-key generation, storage and rotation.
5. Detect weak keys and unauthorised modifications.
6. Support future cryptographic improvements against quantum-computing threats.

The measurable objective is to reduce public-key operations to a small, bounded number during key establishment, while using efficient symmetric encryption for the complete medical file.

Q7.C – Design the Revised Strategy (3 Marks)

The revised architecture combines symmetric encryption, ECC, digital signatures, hybrid encryption and secure key management. This approach provides better performance and security for healthcare communication.

1. Revised Cryptographic Architecture

The proposed architecture consists of the following components:

Identity and Authentication Module

Each clinic and hospital server is provided with a digital certificate. The certificates are verified before communication begins. This ensures that only authorised devices can establish secure connections.

Secure Key Exchange Using ECDH

Elliptic Curve Diffie–Hellman (ECDH) is used to establish a common shared secret between the hospital and remote clinics.

Let the private keys be a and b, and let P be the agreed base point.

The public keys are:

$$A=aP$$

$$B=bP$$

The hospital calculates:

$$K=aB=abP$$

The clinic calculates:

$$K=bA=abP$$

Both parties obtain the same shared secret without directly transmitting their private keys.

Key Derivation Using HKDF

The shared secret is passed through HKDF (HMAC-based Key Derivation Function) to generate secure

session keys. Separate keys are derived for different communication purposes to avoid unnecessary key reuse.

Secure Data Encryption Using AES-GCM

AES-GCM is used to encrypt large medical files and patient records. It provides both confidentiality and integrity.

A unique nonce is generated for every encryption under the same key. An authentication tag is produced to detect unauthorised modifications.

Digital Signature Module

The sender signs important medical records using a private signing key. The receiver verifies the signature using the corresponding public key. This helps confirm the sender's identity and ensures that signed information has not been modified.

Secure Key Management Module

The system maintains secure key generation, storage, distribution, rotation and revocation. Private keys are stored in protected storage, and compromised certificates are revoked to prevent unauthorised access.

2. Security Properties of the Revised Strategy

The proposed techniques provide the following security properties:

- ECDH provides shared-secret establishment.
- HKDF derives separate secure session keys.
- AES-GCM provides confidentiality and integrity.
- Digital signatures provide authentication and integrity evidence.
- Digital certificates verify the identity of communicating devices.
- Key rotation reduces the impact of session-key compromise.

3. Operations Reduced and New Trade-offs

In the original system, applying public-key encryption repeatedly to large data increases computational cost. In the revised system, public-key operations are mainly used for key establishment, while AES-GCM performs the bulk-data encryption.

This reduces the number of expensive public-key operations and improves the efficiency of large-file transfers.

The revised architecture also introduces some additional requirements, such as certificate management, secure key storage, nonce management and session-key rotation. Digital signatures also introduce additional processing overhead.

However, these additional requirements provide stronger authentication, integrity and overall security.

Q7.D – Integrate Quantum Readiness (1 Mark)

Quantum computing may create future security challenges for existing public-key cryptographic algorithms.

RSA depends on the difficulty of factoring large integers, while ECC depends on the difficulty of solving the elliptic-curve discrete logarithm problem. A sufficiently capable fault-tolerant quantum computer running Shor's algorithm could break these mathematical security assumptions.

Quantum Migration Approach

The healthcare network should maintain a cryptographic inventory to identify where RSA and ECC are being used. It should prepare to adopt standardised post-quantum cryptographic algorithms and update certificates, protocols and key-management systems when required.

A hybrid classical and post-quantum key-establishment approach can be considered during the migration period to support compatibility and future security.

Role of Quantum Key Distribution (QKD)

Quantum Key Distribution uses quantum principles to establish shared key material and can help detect eavesdropping under its security assumptions.

However, QKD does not replace digital signatures, endpoint authentication, secure key management or data encryption. It also requires an authenticated classical communication channel and specialised infrastructure.

Therefore, QKD may be considered for selected high-security communication links, but it cannot replace the complete cryptographic architecture.

Q7.E – Predict, Implement and Validate (2 Marks)

1. Prediction Before Implementation

Before implementing the revised architecture, the following improvements are expected:

- The number of public-key operations will be reduced.
- AES-GCM will provide efficient encryption for large medical files.
- Authenticated ECDH will prevent unauthorised key substitution when certificate and handshake verification are correctly implemented.
- Digital signatures will detect modifications to signed medical records.
- Secure key rotation will reduce the risk of long-term session-key exposure.

2. Implementation and Comparison

The original and redesigned systems can be compared using the same synthetic medical data and test environment.

Evaluation factor	**Original system**	**Redesigned system**
Bulk encryption	Direct or repeated RSA operations	AES-GCM
Key exchange	Basic Diffie–Hellman	Authenticated ECDH
Authentication	May be absent	Certificates and signatures

Integrity	Not always provided	AEAD authentication tag
Key management	Limited controls	HKDF and key rotation

3. Validation and Testing

The following tests are performed to verify the proposed system:

1. Encryption and decryption test: Encrypt a medical file and verify that the original file is recovered correctly.
2. Tampering test: Modify the encrypted data and verify that AES-GCM authentication fails.
3. Wrong-key test: Use an incorrect key and verify that decryption is rejected.
4. Digital signature test: Modify the signature and verify that signature validation fails.
5. Certificate validation test: Use an invalid certificate and verify that the connection is rejected.
6. Nonce reuse test: Check that the same nonce is not reused with the same encryption key.
7. Key rotation test: Verify that new sessions use newly generated session keys.

4. Expected Result

The redesigned system is expected to reduce public-key processing and improve the performance of large-file encryption. It also provides better authentication, integrity verification and key-management security.

The actual operation counts and execution times should be recorded by running both implementations using the same input data and hardware. This helps demonstrate the improvement through comparable experimental evidence.

Q7.F – Defend the Final Design (2 Marks)

The proposed hybrid cryptographic architecture is more suitable for the multi-site healthcare network because it combines the advantages of different cryptographic techniques.

Performance: AES-GCM efficiently encrypts large medical files and reduces the need for repeated public-key operations.

Authentication: Authenticated ECDH, digital certificates and signatures help prevent Man-in-the-Middle attacks and verify the identities of communicating devices.

Data Integrity: AES-GCM authentication tags and digital signatures help detect unauthorised modifications to patient records.

Key Management: HKDF, secure random generation, key rotation and certificate revocation improve the protection of cryptographic keys.

Large-File Protection: Hybrid encryption combines efficient symmetric encryption with public-key cryptography for secure key establishment.

Long-Term Security: Cryptographic algorithm flexibility and post-quantum migration planning help prepare the system for future quantum-computing threats.

Residual Risks and Limitations

Although the proposed design improves security, some risks remain:

- A compromised clinic device may expose decrypted patient information.
- Stolen private keys may allow an attacker to impersonate an authorised device.
- Incorrect certificate validation may allow MITM attacks.
- Poor random-number generation may result in weak cryptographic keys.
- Nonce reuse may compromise encryption security.
- Certificate administration and secure storage increase deployment costs.
- Post-quantum migration may require additional software and infrastructure changes.

Final Engineering Decision

The recommended cryptographic strategy is a combination of Authenticated ECDH, HKDF, AES-GCM, Digital Signatures, Certificate-Based Authentication and Secure Key Management.

This architecture reduces expensive public-key operations, improves large-file encryption performance, detects unauthorised modifications and provides stronger authentication between the hospital and remote clinics. It also supports future migration towards post-quantum cryptographic algorithms.

Conclusion

The original cryptographic system has limitations in computational performance, authentication and key management. The proposed redesigned hybrid architecture overcomes these limitations by combining ECDH, HKDF, AES-GCM, digital signatures and certificate-based authentication. It provides efficient bulk-data encryption, secure key exchange, confidentiality, integrity and authentication. With proper implementation, validation and post-quantum migration planning, the proposed strategy offers a secure, efficient and scalable solution for communication between the central hospital and remote healthcare clinics.

7. Final Recommendation

For the multi-site healthcare network, deploy a modern authenticated hybrid cryptographic architecture. Use authenticated ephemeral ECDH for session-key establishment, a KDF for key derivation, AES-GCM or ChaCha20-Poly1305 for large files, and digital signatures for sender identity and audit evidence. Avoid DES, textbook RSA, ECB and unauthenticated DH. Use RSA-OAEP only for compatibility-based key transport, never for bulk files,certificate validation, unique clinic identities, key rotation, revocation, replay protection and protected private-key storage. Maintain crypto agility and prepare a staged transition to post-quantum algorithms.

8. Conclusion

The assignment demonstrates that secure healthcare exchange requires more than selecting a cipher with a large key. Symmetric encryption provides efficient bulk confidentiality, but secure modes and authentication are essential. RSA depends on sound number theory, strong prime generation and secure padding. Diffie–Hellman establishes a shared secret but requires authentication to prevent MITM. ECC provides compact public-key material for constrained devices. Digital signatures, AEAD and hybrid encryption together provide confidentiality, integrity and sender authenticity for large files.

9. References

- William Stallings, Cryptography and Network Security: Principles and Practice, Pearson.
- Jonathan Katz and Yehuda Lindell, Introduction to Modern Cryptography, CRC Press.
- National Institute of Standards and Technology (NIST), FIPS 186-5: Digital Signature Standard.
- NIST, FIPS 203: Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM).
- NIST, FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).
- NIST, SP 800-56A, Recommendation for Pair-Wise Key-Establishment Schemes Using Discrete Logarithm Cryptography.
- NIST, SP 800-38D, Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM).
- RFC 8017, PKCS #1: RSA Cryptography Specifications Version 2.2.
- RFC 8446, The Transport Layer Security (TLS) Protocol Version 1.3.

Appendix A – Core Python routines for mathematical verification

The following routines are compact educational implementations for the arithmetic tasks. They should be tested with the toy values in the report.

```
def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def egcd(a, b):
    if b == 0:
        return a, 1, 0
    g, x1, y1 = egcd(b, a % b)
    return g, y1, x1 - (a // b) *
    y1

def inverse(a, m):
    g, x, _ = egcd(a,
    m) if g != 1:
        raise ValueError("No
    inverse") return x % m

def power_mod(base, exponent,
    modulus): result = 1
    base %=
    modulus while
    exponent:
        if exponent & 1:
            result = (result * base) %
        modulus base = (base * base) %
        modulus exponent >>= 1
    return result

# RSA toy test
p, q, e = 61, 53, 17
n = p * q
```

phi = (p - 1) * (q -
1) d = inverse(e,
phi) message = 65
cipher = power_mod(message,
e, n) recovered =
power_mod(cipher, d, n) assert
recovered == message print(n,
phi, d, cipher, recovered)
test p, g, a, b = 23, 5,
6, 15
A = power_mod(g, a, p)
B = power_mod(g,
b, p) KA =
power_mod(B, a, p)
KB =
power_mod(A, b,
p) assert KA == KB
print(A, B, KA,
KB)

Experiment	Input / parameters	Operations counted	Measured time	Verified result
Symmetric encryption	1 KB / 100 KB / 1 MB / 10 MB	Blocks and rounds	To be measured	Pass / Fail
Modular exponentiation	Same base/modulus; varied exponent size	Modular multiplications	To be measured	Pass / Fail
Primality tests	Prime and composite toy candidates	Divisions / rounds	To be measured	Correct classificatio n
RSA toy test	n=3233, e=17, M=65	Modular multiplications	To be measured	65 recovered
DH toy test	p=23, g=5, a=6, b=15	Modular multiplications	To be measured	Shared value 2
Hybrid file test	1 KB / 100 KB / 1 MB / 10 MB	Public-key ops + AEAD bytes	To be measured	Tampering rejected